

# AI Media Studio

# User Manual

A fully local, AI-powered pipeline that turns one topic into a finished, narrated, subtitled short-form video. No cloud services and no paid APIs.

|          |                                    |
|----------|------------------------------------|
| Version  | 0.1.0                              |
| Requires | Python 3.12 or newer               |
| Runs on  | Windows, verified on Windows 11    |
| Output   | 1080x1920 H.264 MP4 with AAC audio |

## Contents

- 1 What this does
- 2 Before you start
- 3 Installing
- 4 Configuration
- 5 Generating a video
- 6 Resuming a run
- 7 Inside a project folder
- 8 Scenes, motion and transitions
- 9 Animating scenes with Stable Video Diffusion
- 10 How timing works
- 11 Background music
- 12 Setting up the ComfyUI workflow
- 13 Troubleshooting
- 14 Running the tests

# 1 What this does

You give AI Media Studio a topic. It researches and writes a script, breaks it into ordered scenes, narrates them, generates an image for each scene, writes subtitles, and renders the whole thing into a vertical MP4 ready for Reels, Shorts or TikTok. Everything runs on your own machine.

## The pipeline

|                                         |                      |
|-----------------------------------------|----------------------|
| Topic                                   |                      |
| -> Script and ordered scenes            | (Ollama)             |
| -> Narration, measured word by word     | (Kokoro)             |
| -> Scene durations reconciled to speech |                      |
| -> One image per scene                  | (ComfyUI)            |
| -> One clip per scene, optional         | (ComfyUI, section 9) |
| -> Subtitles                            | (SRT)                |
| -> Final MP4                            | (FFmpeg)             |

Each stage writes its results to a project folder as it goes. Nothing is held only in memory, which is what makes an interrupted run resumable (section 6).

## What you need to supply

- A topic, as a short phrase.
- Optionally a style, a voice, background music, and whether you want subtitles.

# 2 Before you start

Four local services do the actual work. Install and start these first.

| Component | Purpose                                                   | Notes                                                                                        |
|-----------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Ollama    | Writes the script and plans scenes                        | Must be running. Pull a model first, for example <code>ollama pull llama3.1:8b</code> .      |
| ComfyUI   | Generates one image per scene, and optionally animates it | Must be running, with an image model installed. See section 12, and section 9 for animation. |
| Kokoro    | Speaks the narration                                      | Installed automatically as a Python dependency. Downloads its voice on first use.            |
| FFmpeg    | Renders the final video                                   | Either on your PATH or given as a full path in the configuration.                            |

## Hardware

Image generation is the demanding part. A machine with an 8 GB NVIDIA GPU produces an SDXL image in roughly 20 to 35 seconds once the model is loaded in memory, and 90 seconds or more before that. Less video memory still works but is slower, because the model is reloaded for each image.

# 3 Installing

```
pip install -e .
copy config\settings.example.toml config\settings.toml
```

Then open `config\settings.toml` and replace the two placeholder values: the Ollama model name and the Kokoro voice. Nothing will run until you do, because the placeholders are not real model names.

**Installing without -e?** A non-editable install has no project folder to read from, so tell it where your settings file is with `--config` or the `AI_MEDIA_CONFIG` environment variable, and use absolute paths inside that file.

## Checking it worked

```
ai-media-studio generate --help
```

## 4 Configuration

All settings live in one TOML file. By default that is `config\settings.toml` inside the project folder.

### Choosing a different file

Highest priority first:

- The `--config` option on the command line.
- The `AI_MEDIA_CONFIG` environment variable.
- The project's own `config\settings.toml`.

### The settings that matter most

| Setting                                        | What it controls                                                  | Default                      |
|------------------------------------------------|-------------------------------------------------------------------|------------------------------|
| <code>ollama.model</code>                      | Which local language model writes the script                      | none, you must set it        |
| <code>kokoro.voice</code>                      | Which narration voice is used                                     | none, you must set it        |
| <code>kokoro.speed</code>                      | Speaking rate                                                     | 1.0                          |
| <code>kokoro.scene_tail_padding_seconds</code> | Silence after each scene so speech does not butt against a cut    | 0.2                          |
| <code>paths.ffmpeg_executable</code>           | FFmpeg command or full path                                       | ffmpeg                       |
| <code>paths.output_dir</code>                  | Where project folders are created                                 | output                       |
| <code>comfyui.workflow_path</code>             | The ComfyUI workflow JSON to run                                  | config/comfyui_workflow.json |
| <code>comfyui.timeout_seconds</code>           | How long to wait for one image                                    | 120                          |
| <code>comfyui.max_retries</code>               | Retries after a recoverable image failure                         | 2                            |
| <code>video.width, video.height</code>         | Output resolution                                                 | 1080 by 1920                 |
| <code>video.frames_per_second</code>           | Output frame rate                                                 | 30                           |
| <code>video.transition_duration_seconds</code> | How long a transition lasts                                       | 0.5                          |
| <code>video.animate_still_scenes</code>        | Move the image even when the storyboard asks for no camera motion | true                         |

| Setting                          | What it controls                                                      | Default   |
|----------------------------------|-----------------------------------------------------------------------|-----------|
| video.camera_motion_strength     | How far a zoom or pan travels                                         | 0.2       |
| video.render_timeout_seconds     | How long to wait for the render                                       | 300       |
| video.clip_smoothing             | How the frames a slowed clip lacks are made. Only used with section 9 | blend     |
| svd.enabled                      | Animate each scene image instead of panning across it                 | false     |
| svd.frames, svd.fps              | Length of a generated clip, as frames over frames per second          | 25 over 6 |
| svd.motion_bucket_id             | How much a clip moves, at the cost of coherence                       | 127       |
| subtitles.max_characters_per_cue | Longest caption shown at once                                         | 32        |
| gpu.device                       | Device for models running in this process, meaning narration          | auto      |
| music.volume                     | Background music level                                                | 0.15      |
| music.ducking_ratio              | How hard music dips under narration                                   | 6.0       |
| logging.level                    | Detail written to the log                                             | INFO      |

**Raise** `comfyui.timeout_seconds` **to about 300**. The very first image of a run also has to load the image model into memory, which alone can take longer than the 120 second default. Otherwise the first scene times out and is retried needlessly.

## Overriding settings without editing the file

Any of these environment variables takes precedence over the file, which is convenient for one-off changes and for scripting.

|                                            |                                       |
|--------------------------------------------|---------------------------------------|
| AI_MEDIA_CONFIG                            | AI_MEDIA_LOG_LEVEL                    |
| AI_MEDIA_OLLAMA_BASE_URL                   | AI_MEDIA_LOG_DIRECTORY                |
| AI_MEDIA_OLLAMA_MODEL                      | AI_MEDIA_LOG_CONSOLE_ENABLED          |
| AI_MEDIA_OLLAMA_TIMEOUT_SECONDS            | AI_MEDIA_LOG_FILE_ENABLED             |
| AI_MEDIA_COMFYUI_BASE_URL                  | AI_MEDIA_GPU_DEVICE                   |
| AI_MEDIA_COMFYUI_WORKFLOW_PATH             | AI_MEDIA_VIDEO_WIDTH                  |
| AI_MEDIA_COMFYUI_TIMEOUT_SECONDS           | AI_MEDIA_VIDEO_HEIGHT                 |
| AI_MEDIA_KOKORO_VOICE                      | AI_MEDIA_VIDEO_FPS                    |
| AI_MEDIA_KOKORO_SPEED                      | AI_MEDIA_VIDEO_RENDER_TIMEOUT_SECONDS |
| AI_MEDIA_VIDEO_TRANSITION_DURATION_SECONDS |                                       |

Settings without a variable in this list, including everything under `[svd]` and `[subtitles]`, are set in the file.

## 5 Generating a video

```
ai-media-studio generate --topic "Why Japan Never Sleeps"
```

With every option in use. The trailing backtick continues a command across lines in PowerShell; in Command Prompt use a caret instead.

```
ai-media-studio generate `
  --topic "Why Japan Never Sleeps" `
  --style "cinematic documentary" `
  --voice "af_heart" `
  --output "D:\Media Projects" `
  --music `
  --subtitle
```

## Options

| Option     | Meaning                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------|
| --topic    | The subject of the video. Required unless you are resuming.                                         |
| --resume   | Continue an existing project folder instead of starting fresh. Cannot be combined with --topic.     |
| --style    | Narrative and visual style, for example <i>cinematic documentary</i> . Passed to the script writer. |
| --voice    | Override the narration voice for this run only.                                                     |
| --output   | Where the project folder is created, overriding the configured output folder.                       |
| --config   | Use a different settings file.                                                                      |
| --music    | Pick a random track from your music folder and mix it under the narration.                          |
| --subtitle | Write subtitles and burn them into the video.                                                       |

## What you will see

```
[#####-----] 1/3 Generating storyboard, narration, and images
[#####-----] 2/3 Generating subtitles
[#####-----] 3/3 Rendering final MP4

Generation complete
Generation time: 505.94 seconds
Provider versions: ollama=unknown
Final output: C:\AI-Media-Studio\output\20260808T164700953894Z_why-japan-never-sleeps\video\final.mp4
```

## Roughly how long it takes

Measured on a laptop with an 8 GB NVIDIA GPU, for a six scene video.

| Stage     | Typical time          | Notes                                         |
|-----------|-----------------------|-----------------------------------------------|
| Script    | 15 to 60 seconds      | Depends on the language model and topic.      |
| Narration | 80 to 115 seconds     | Slower the first time, while the voice loads. |
| Images    | 20 to 95 seconds each | The first two are slowest, then it speeds up. |

| Stage     | Typical time         | Notes                                      |
|-----------|----------------------|--------------------------------------------|
| Clips     | about 4 minutes each | Only when [svd] is enabled. See section 9. |
| Subtitles | under a second       | Written from measured word timings.        |
| Rendering | 5 to 25 seconds      | Longer with music, subtitles and clips.    |

A whole run is typically 6 to 10 minutes, and almost all of it is image generation. Enabling animation (section 9) takes it to roughly half an hour. If a run fails near the end, resume it rather than starting over.

## 6 Resuming a run

Every stage saves its work as it finishes, so a run that stops partway through can carry on. This matters most when the image service becomes unavailable mid-run, which otherwise throws away a finished script and narration.

```
ai-media-studio generate --resume "output\20260808T164700953894Z_why-japan-never-sleeps" --subtitle
```

When resuming:

- The script is read back from the project folder and never regenerated.
- Existing narration is reused as it is, along with the word timings recorded when it was spoken, so captions still follow the narration word by word.
- Scene images already on disk are kept, and only missing ones are generated.
- Animated clips are kept too. They are the most expensive thing a run produces, so they are never made twice.
- The video is always rendered again, which is quick.

Resuming a project that is already complete regenerates nothing and simply re-renders, so it is a cheap way to try different music or subtitle settings without paying for image generation again.

To regenerate one artifact, delete it and resume. Removing `narration.wav` re-speaks the script and re-times every scene to it; removing a single file from `images\` regenerates only that scene.

## 7 Inside a project folder

Each run creates one timestamped folder under your output folder, named after the time and the topic.

```

20260808T164700953894Z_why-japan-never-sleeps\
manifest.json      what ran, which providers, how long, and whether it succeeded
storyboard.json    the script and every scene, with their final durations
narration.wav      the spoken narration for the whole video
word_timings.json  when each word is spoken, which the captions are built from
subtitles.srt      subtitle cues, when --subtitle was used
images\
  scene_001.png    one image per scene
  scene_002.png
clips\
  scene_001.webm   one animated clip per scene, when [svd] is enabled
video\
  final.mp4        the finished video
logs\

```

**manifest.json** is the place to look when something went wrong. It records which stage failed and why, and is written even for a failed run.

**storyboard.json** holds the narration text and the scene timings actually used. If you want to know why a video is the length it is, read this.

**word\_timings.json** is written when the narration is spoken, because only the voice can measure it. Deleting it does not break anything, but captions then fall back to one cue per scene, a paragraph at a time.

## 8 Scenes, motion and transitions

The script writer assigns each scene a camera motion and a transition into the next one. Only the values below are accepted; anything else the model invents is converted to the closest match, so an unusual word never spoils a run.

### Camera motion

| Value          | Effect                                |
|----------------|---------------------------------------|
| none           | The image is held still.              |
| zoom_in        | Slow push in towards the centre.      |
| zoom_out       | Slow pull back from the centre.       |
| pan, pan_right | Drifts across the image to the right. |
| pan_left       | Drifts across the image to the left.  |

### Transitions

| Value               | Effect                                     |
|---------------------|--------------------------------------------|
| cut, none           | An instant change with no blend.           |
| fade, crossfade     | One scene blends into the next.            |
| dissolve            | A softer, more textured blend.             |
| wipeleft, wiperight | The next scene slides across horizontally. |
| wipeup, wiptdown    | The next scene slides across vertically.   |

Transition length comes from `video.transition_duration_seconds`. A transition is automatically shortened if either neighbouring scene is too brief to accommodate it.

**Language models almost always ask for no camera motion**, which leaves every image frozen on screen. Scenes asking for none are therefore given a movement anyway, alternating between zoom and pan so consecutive scenes differ. Set `video.animate_still_scenes` to false to follow the storyboard exactly.

## The opening line

The script writer names a hook for the opening seconds. It leads the first scene's narration, so it is spoken and captioned like any other line. The merge happens once, before the storyboard is saved, so resuming a run does not speak it twice, and a hook the model already used as its opening line is left alone rather than repeated.

A model asked only for a hook writes atmospheric scene setting, so the script prompt says what the hook is for: one sentence, at most twelve words, stating a fact, question or claim rather than describing the view.

**Read the opening line before you publish.** The prompt asks for an accurate hook and the model still occasionally overstates one, because nothing checks a claim against reality. Automatic fact checking is not built yet.

## Subtitles

Captions follow the narration word by word rather than showing a whole scene's text at once. The voice provider records when each word is spoken while it synthesizes the speech, and those timings drive the cues. Sentences are divided into cues of roughly equal length, so none is left holding a single word, and each stays on screen until the next begins.

Those timings are saved to the project folder, so resuming a run keeps captions word by word rather than dropping back to one cue per scene.

# 9 Animating scenes with Stable Video Diffusion

By default each scene is a still image with a slow pan or zoom across it. Turn this on and the picture itself moves: rain falls, light flickers, and clouds drift. Each scene is generated twice, once as a still image and again as a short clip animating that image.

This is off by default because it is slow and needs another model. It uses the same ComfyUI you already have.

## Setting it up

- Put a Stable Video Diffusion checkpoint in ComfyUI's `models\checkpoints` folder, named by `config\svd_workflow.json`.
- Set `svd.enabled` to true.

```
huggingface-cli download stabilityai/stable-video-diffusion-img2vid-xt \
  svd_xt.safetensors --local-dir comfyui\models\checkpoints
```



## What it costs

Expect three to four minutes per scene on an 8 GB card, so a five scene video takes around half an hour rather than eight minutes. Clips are saved in the project folder and reused when a run is resumed, because they are by far the most expensive thing a run produces.

On a card with less memory than the model needs, the model is reloaded for every single clip and the time per scene barely improves as the run goes on. This is normal and not a fault.

## Why clips are slowed, and what that needs

A clip runs `svd.frames` divided by `svd.fps` seconds, about four by default, while a scene lasts as long as its narration. Clips are therefore slowed to fill their scene.

Slowing 25 frames across a ten second scene would leave each picture on screen for four tenths of a second, which looks like stuttering rather than slow motion. The frames in between are generated instead, controlled by `video.clip_smoothing`.

| Value  | Cost per scene     | Result                                                                                                                                                     |
|--------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| blend  | a few seconds      | Each frame fades into the next. The default, and hard to tell from <i>motion</i> on the soft drifting movement these clips usually have.                   |
| motion | one to two minutes | Movement between frames is followed, which is sharper when a clip has a clearly moving subject. Raise <code>video.render_timeout_seconds</code> to use it. |
| none   | none               | Frames are repeated, which stutters. Included for comparison.                                                                                              |

Camera motion is not applied to a scene that has a clip, because the picture already moves. A scene whose animation fails falls back to its still image, so a failure costs quality rather than the whole run.

## 10 How timing works

This is worth understanding, because it explains why your video is the length it is.

The language model estimates how long each scene should take to say. Those estimates are unreliable and are never used to time the video. Instead the narration for each scene is generated first and measured, and those measurements become the timeline.

The result is that:

- The video is exactly as long as the narration. There is no trailing silence, and narration is never cut off mid-sentence.
- Each image is on screen for exactly as long as its own narration.
- Each transition begins the moment its scene stops speaking.
- Subtitles line up with the speech, because they use the same measurements.

A consequence worth knowing: **you cannot set the video length directly**. It is whatever the narration takes. To get a shorter video, ask for a shorter script or raise `kokoro.speed`.

## 11 Background music

Put audio files in the folder named by `music.directory`, which is `music\` by default. Accepted formats are mp3, wav, m4a, aac, flac and ogg.

With `--music`, one track is chosen at random, looped to fit, faded in and out, and automatically dipped underneath the narration so the voice stays clear. Adjust the level with `music.volume` and the strength of the dip with `music.ducking_ratio`.

If the folder is missing or empty you get a warning and the video is still produced, just without music.

## 12 Setting up the ComfyUI workflow

Image generation runs whatever ComfyUI workflow you point it at, so you can use any model and any settings ComfyUI supports. Export your workflow from ComfyUI in API format and set `comfyui.workflow_path` to it.

Your workflow must satisfy two rules:

- Exactly one positive prompt node, found by following the sampler's positive input. The scene's image prompt is written into it for each image.
- Exactly one Save Image node, which is where the finished image is collected.

If either is missing or ambiguous you get a clear configuration error naming the problem, rather than a failed render later on. Nothing else about the workflow matters, so resolution, sampler, steps and model are entirely yours to choose.

Generate portrait images that match your output aspect ratio, for example 768 by 1344 for a 1080 by 1920 video. Images are scaled and cropped to fit, so a landscape image will lose a lot of its edges.

## 13 Troubleshooting

The log file, by default `logs\ai_media_studio.log`, records every stage with timings and the reason for any failure.

| Message you see                                         | What it means and what to do                                                                                                                                    |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Configuration file not found                            | There is no settings file where it looked. Copy the example file to <code>config\settings.toml</code> , or pass <code>--config</code> .                         |
| Generation failed during storyboard: HTTP Error 404     | Ollama is running but does not have the model named in your settings. Check <code>ollama list</code> and pull the model, or correct <code>ollama.model</code> . |
| Generation failed during storyboard: connection_failed  | Ollama is not running, or is on a different address than <code>ollama.base_url</code> .                                                                         |
| Generation failed during image: Unable to reach ComfyUI | ComfyUI is not running or has stopped. Start it, then resume the run with <code>--resume</code> so the script and narration are not lost.                       |
| ComfyUI generation timed out                            | Usually the first image of a run, while the model loads. Raise <code>comfyui.timeout_seconds</code> to about 300.                                               |
| No positive prompt node could be discovered             | Your workflow has no prompt node reachable from a sampler's positive input, or has several. See section 12.                                                     |
| Multiple Save Image output nodes were discovered        | Your workflow saves more than one image. Leave exactly one Save Image node.                                                                                     |

| Message you see                                             | What it means and what to do                                                                                                                                                                                                                |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Generation failed during render: FFmpeg is unavailable      | FFmpeg is not on your PATH. Set <code>paths.ffmpeg_executable</code> to its full path.                                                                                                                                                      |
| Generation failed during render: FFmpeg rendering timed out | Raise <code>video.render_timeout_seconds</code> . Generating the frames between a clip's own frames is the slow part, especially with <code>video.clip_smoothing</code> set to <i>motion</i> .                                              |
| Generation failed during clip                               | ComfyUI could not animate a scene. Check that the Stable Video Diffusion checkpoint named in <code>config\svd_workflow.json</code> is actually in ComfyUI's checkpoints folder. Resume the run afterwards; the clips already made are kept. |
| Narration runs N seconds but the scenes span M seconds      | The narration and the scene list no longer agree, which happens if one is regenerated without the other. Delete <code>narration.wav</code> and resume, so both are rebuilt together.                                                        |
| Music warning, or Subtitle warning                          | Not a failure. The video is still produced without that extra. Usually an empty or missing music folder.                                                                                                                                    |

## Image generation is unusually slow

If every image takes 90 seconds rather than settling to around 25, the image model is being reloaded for each one because memory is tight. Close other applications, or use a smaller image model. Speed usually improves a few scenes into a run.

## The picture jumps rather than moving smoothly

If animated scenes look like they are lagging, check that `video.clip_smoothing` is not set to *none*. A clip holds about twenty five frames, so filling a long scene by repeating them leaves each picture on screen for a noticeable moment. See section 9.

## ComfyUI stops on its own during long runs

On machines with limited memory the image service can exit without an error. Restart it and resume the run, which keeps everything already generated.

# 14 Running the tests

The test suite uses only the Python standard library and never contacts a local model, so it runs in well under a second and is safe to run at any time.

```
python -m unittest discover -s tests -t .
```

Use it to confirm an installation is sound before spending time on a full run.